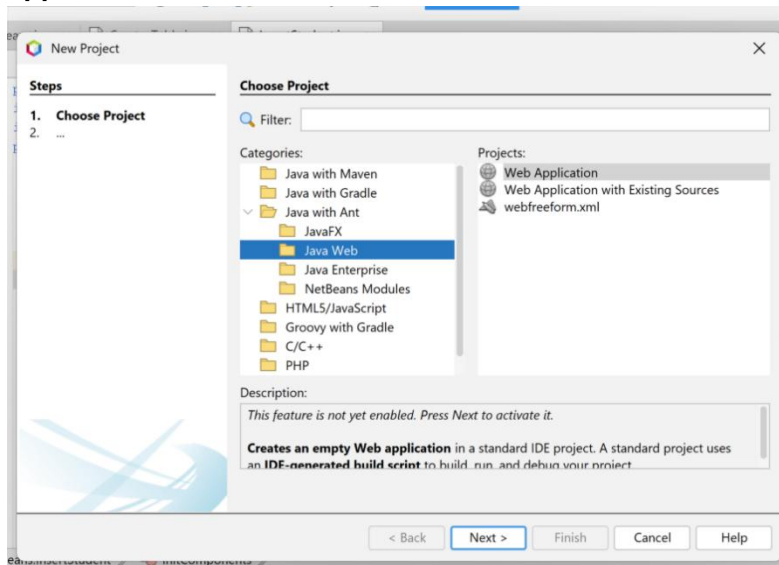


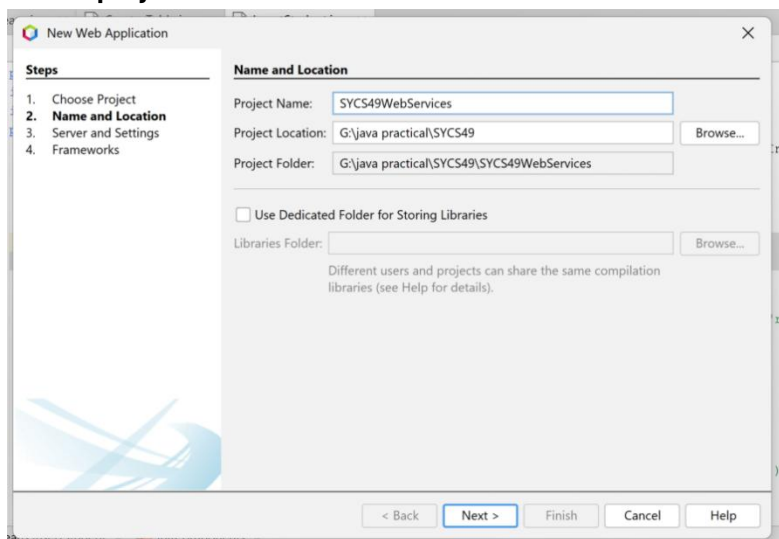
Aim: Web Services.

Steps to create Web Application project.

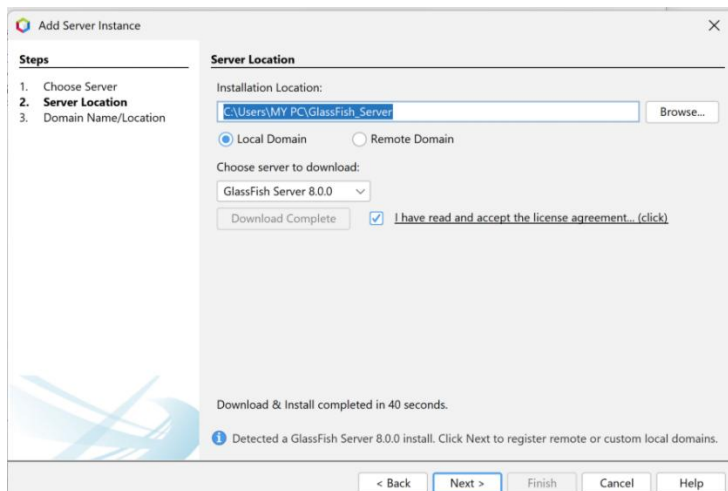
1. In new project in categories inside Java with Ant select java web and select Web applications then click on next.

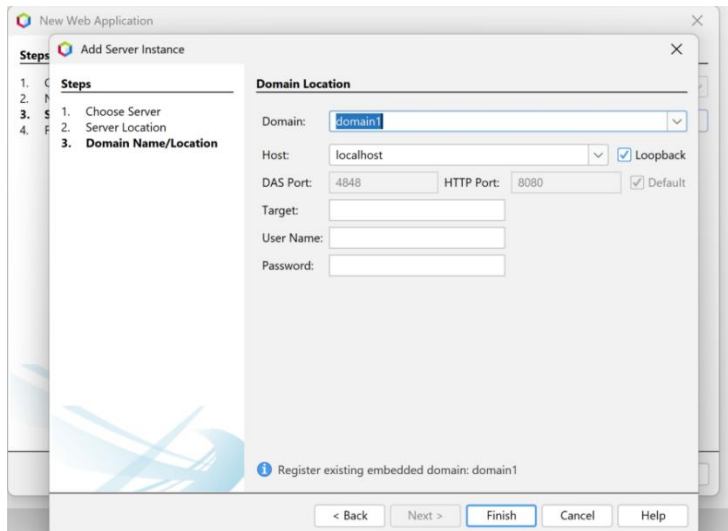
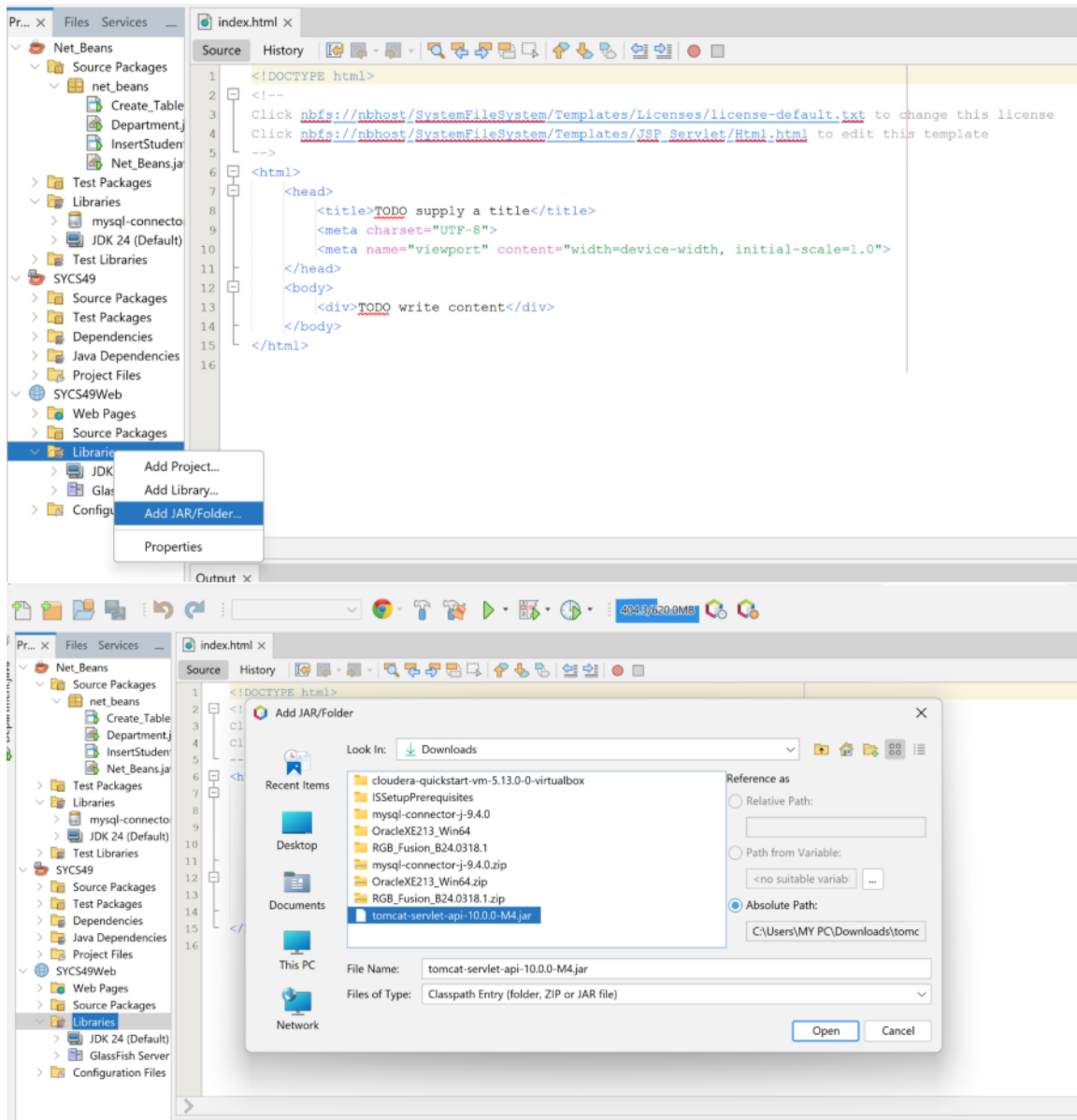


2. Give project name then click on next.

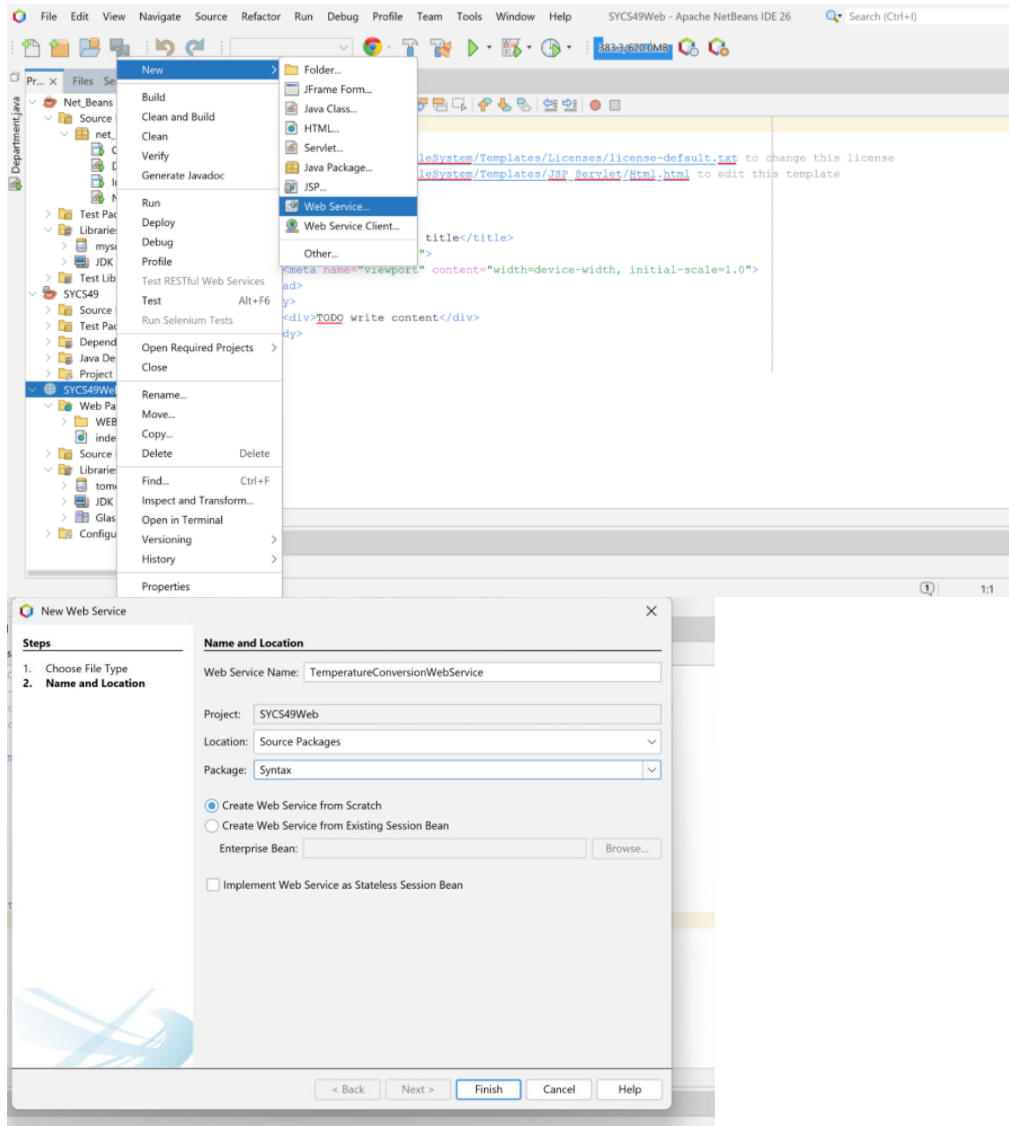


3. Choose GlassFish Server from choose server to download then click on next.

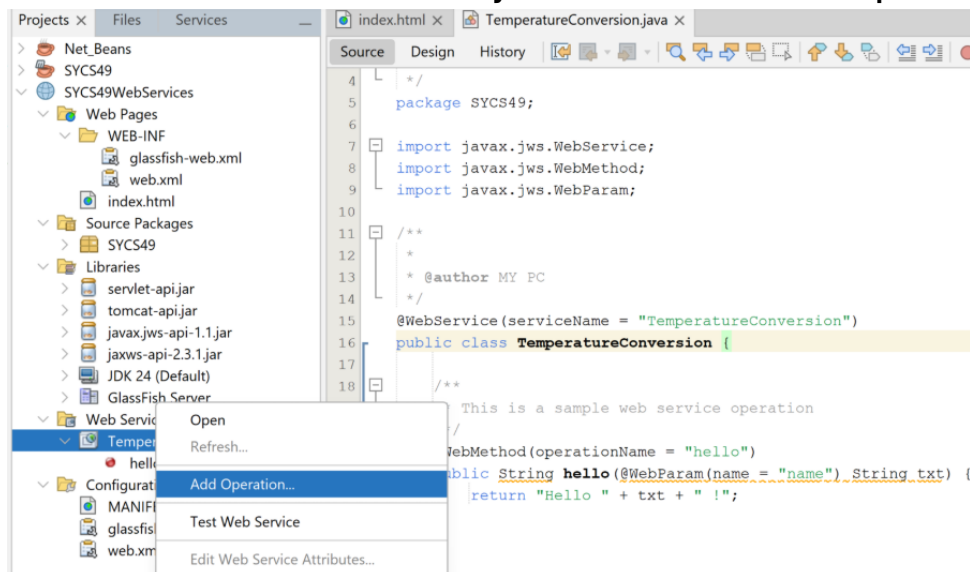


4. Click on finish.**5. Now your web server is created. After that right click on libraries and select Add Jar/Folder to add required libraries.**

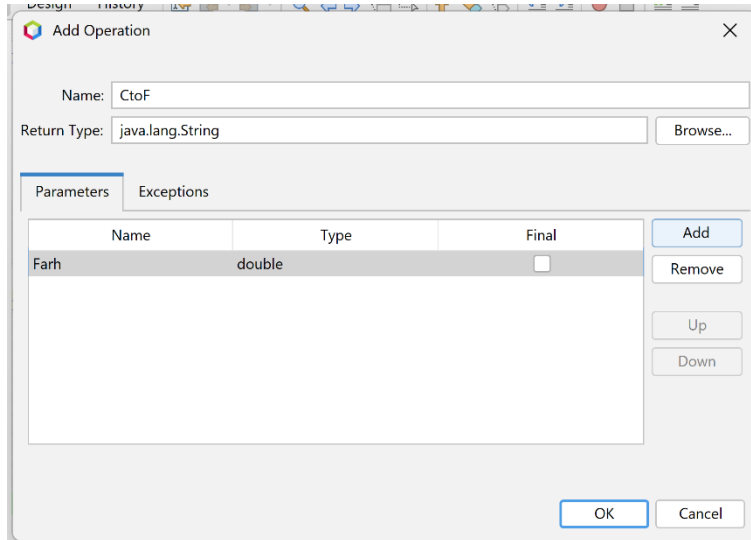
6. Now add new web service.



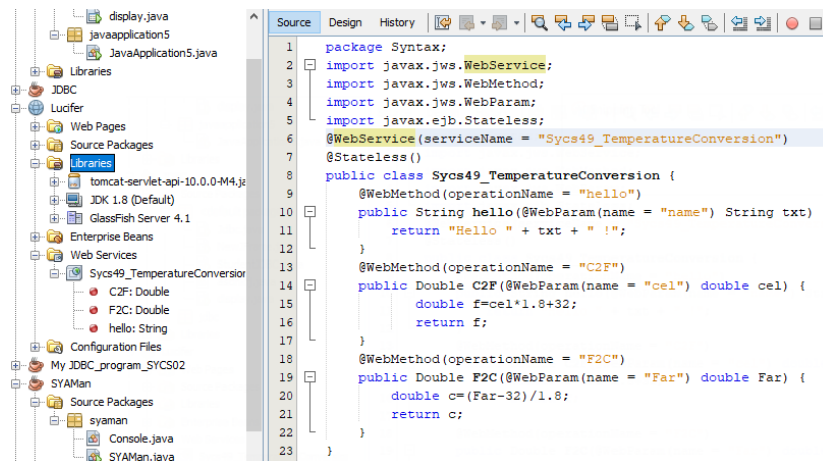
7. Write click on the web service that you created and select Add operation.



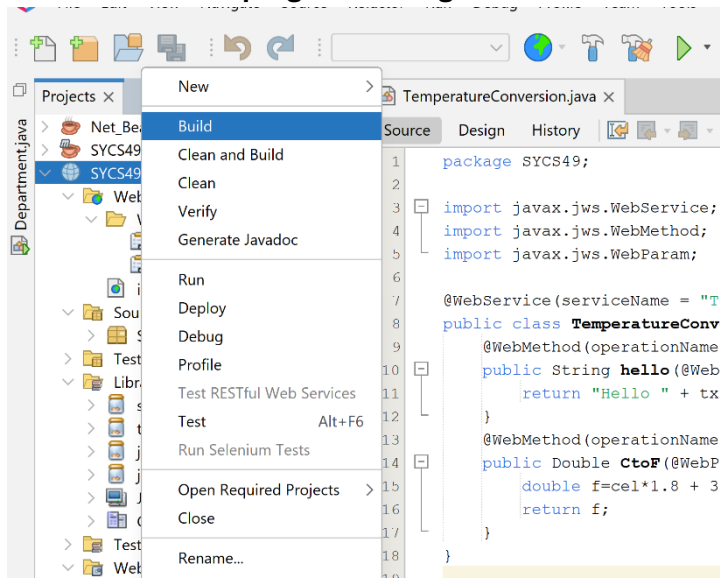
8. Give the return type and add parameter according to your requirement and click on OK.



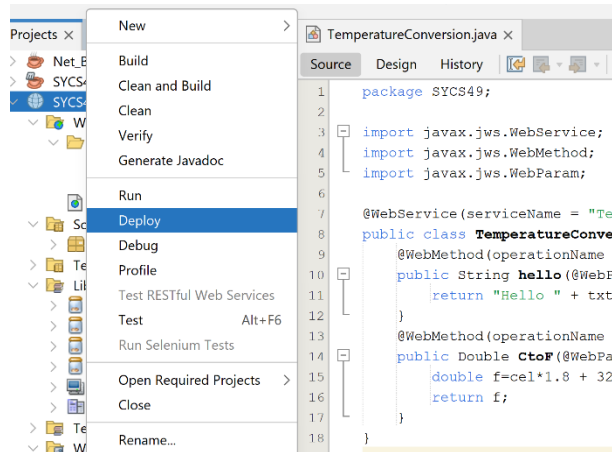
9. Finally write a program for Temperature conversion.



10. To execute the program first right click on the web server and then click on build.



11. Again right click on your web server now select Deploy.



12. Finally run the program you will get your output.

OUTPUT:

SyCS49_TemperatureConversion Web Service Tester

This form will allow you to test your web service implementation ([WSDL File](#))

To invoke an operation, fill the method parameter(s) input boxes and click on the button labeled with the method name.

Methods :

public abstract java.lang.Double syntax.SyCS49TemperatureConversion.f2C(double)

f2C (32)

public abstract java.lang.Double syntax.SyCS49TemperatureConversion.c2F(double)

c2F (0)

public abstract java.lang.String syntax.SyCS49TemperatureConversion.hello(java.lang.String)

hello ()

f2C Method invocation

Method parameter(s)

Type	Value
double	32

Method returned

java.lang.Double : "0.0"

2. Write a web service for conversation of dollar to rupees.

CODE:

```
package webpack;
import javax.xml.ws.WebService;
import javax.xml.ws.WebMethod;
import javax.xml.ws.WebParam;
import javax.xml.ws.Stateless;
@WebService(serviceName = "DollarToRupees")
@Stateless()
public class DollarToRupees {
```

```
@WebMethod(operationName = "hello")
public String hello(@WebParam(name = "name") String txt) {
    return "Hello " + txt + " !";
}

/**
 * Web service operation
 */
@WebMethod(operationName = "dollar_to_rupee")
public Double dollar_to_rupee(@WebParam(name = "Doller") double dollar) {
    //TODO write your implementation code here:
    double rup = dollar*84.57;
    return rup;
}
}
```

OUTPUT:

DollarToRupees Web Service Tester

This form will allow you to test your web service implementation ([WSDL File](#))

To invoke an operation, fill the method parameter(s) input boxes and click on the button labeled with the method name.

Methods :

public abstract java.lang.String webpack.DollarToRupees.hello(java.lang.String)
hello (Atomic)

public abstract java.lang.Double webpack.DollarToRupees.dollarToRupee(double)
dollarToRupee (55)

dollarToRupee Method invocation

Method parameter(s)

Type	Value
double	55

Method returned

java.lang.Double : "4651.349999999999"

3. Write a web service for finding today's temperature.**CODE:**

```
import javax.xml.ws.WebService;  
import javax.xml.ws.WebMethod;  
@WebService(serviceName = "TodayTemp")
```

```
public class TodayTemp {  
    @WebMethod(operationName = "TTemp")  
    public String TTemp() {  
        return "Today's temperature is 32°C";  
    }  
}
```

OUTPUT:**Methods :**

```
public abstract java.lang.String mywebpack.TodayTemp.tTemp()  
tTemp ()
```

tTemp Method invocation**Method parameter(s)**

Type	Value
------	-------

Method returned

java.lang.String : "Today's temperature is 32°C"

4. Write a web service for finding today's Gold rate.**CODE:**

```
import javax.xml.ws.WebService;  
import javax.xml.ws.WebMethod;  
  
@WebService(serviceName = "GoldRate")  
public class GoldRate {  
    @WebMethod(operationName = "CurrentGoldRate")  
    public String CurrentGoldRate() {  
        return "Today's Gold Rate: ₹6000 per gram";  
    }  
}
```

OUTPUT:**Methods :**

```
public abstract java.lang.String mywebpack.GoldRate.currentGoldRate()  
currentGoldRate ()
```

currentGoldRate Method invocation**Method parameter(s)**

Type	Value
------	-------

Method returned

java.lang.String : "Today's Gold Rate: ₹6000 per gram"